# EE542 Lab 7: Artificial Neural Networks, from NumPy to a Cloud GPU

**Huacong Tang**
University of Southern California

**Xiao Yang**
University of Southern California

**Jiachen Lu**
University of Southern California

Code Demo Video

## 1 A two-layer MLP in NumPy

We implemented the tutorial's two-layer perceptron (ReLU hidden layer, MSE or softmax cross-entropy output, mini-batch SGD with backpropagation) in NumPy and ran its three examples with the tutorial's data and hyperparameters (Figure 1). The $1-32-1$ regressor fits the piecewise-linear target with an RMSE of 0.032 against the noise-free function, below the 0.05 noise it was trained on. On the concentric rings the $2-64-2$ classifier reaches 99.9% training accuracy and 99.5% on a fresh draw from the same generator; on the three Gaussian blobs, 95.1% and 95.4%. Each example trains in under 0.2 s. We ran the script on a Mac mini (Apple M4) and on the cloud instance of Section 3, and every loss and accuracy agreed to the printed digit.

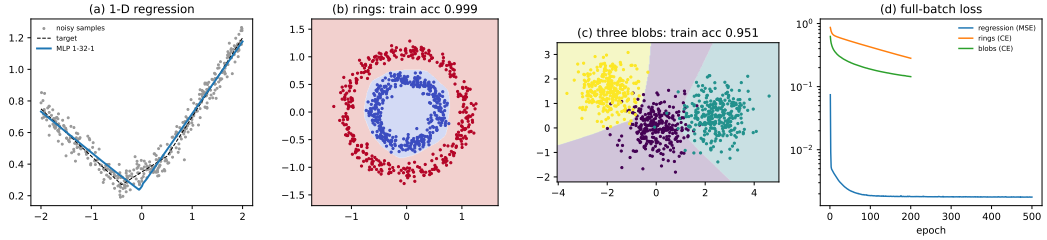



Figure 1: The three examples: (a) regression fit, (b, c) decision regions, (d) full-batch loss per epoch.

## 2 A one-filter CNN in NumPy

**Data.** Kaggle's Dogs vs. Cats images come from Microsoft's Asirra set, which Microsoft distributes directly as 12,500 cat and 12,500 dog JPEGs; we used that copy because it needs no Kaggle account. Two files do not decode, leaving 24,998. Kaggle's own test folder has no labels, so as in the tutorial we hold out 20% of the labeled images: 19,998 for training and 5,000 for testing, balanced. We shuffle before splitting. The tutorial's `load_data` returns all cats and then all dogs, so cutting that array at 80% would put almost only dogs in the test set.

**Model and fixes.** The model is the tutorial's: one random $3 \times 3 \times 3$ filter, ReLU, $2 \times 2$ max-pooling, and a fully connected layer with a ReLU output, trained image by image with MSE loss, learning

rate $10^{-3}$, for 10 epochs. Only the fully connected layer is trained. Two changes were needed for the code to run and mean something. The filter turns a $64 \times 64$ image into $62 \times 62$, and pooling makes that $31 \times 31$, so the weight matrix must be $961 \times 1$; the tutorial's $3072 \times 1$ fails in `np.dot`. We also shuffle the training images once, since image-by-image SGD over 10,000 cats followed by 10,000 dogs ends every epoch having just seen only dogs.

**Evaluation and timing.** The script trains on the training folder and then classifies the 5,000 unseen images in five blocks of 1,000, timing each block from pixels to prediction (Table 1). The tutorial's double-loop convolution takes 13.2 ms per image, so the test set runs at 75 images per second. Because the filter is never trained, its output for an image is the same in every epoch, and the tutorial's loop recomputes it ten times: 10 epochs cost 2,647 s, of which the fully connected layer is 2.7 s. We compute the features once and reuse them. To check that this is the same computation, we also ran the tutorial's loop literally for one epoch (270 s), and its weights matched ours exactly. Writing the same convolution as nine shifted multiply-adds over a batch takes 0.11 ms per image, 120 times faster, with identical accuracy at the same seed.

| Conv | Output | $b_0$ | Conv (ms/img) | Train, 10 ep. (s) | Test (img/s) | Dead at init (%) | Test acc. (%) |
|---|---|---|---|---|---|---|---|
| Loop (tutorial) | ReLU | 0 | 13.223 | 2647 | 75 | 99.8 | 53.5 |
| Vectorized | ReLU | 0 | $0.110 \pm 0.000$ | $25 \pm 0$ | $4039 \pm 87$ | $72.2 \pm 45.6$ | 50.0–56.5 |
| Vectorized | ReLU | 0.5 | $0.110 \pm 0.000$ | $25 \pm 0$ | $4078 \pm 29$ | $0.7 \pm 1.2$ | 50.0–56.5 |
| Vectorized | linear | 0 | $0.113 \pm 0.002$ | $25 \pm 0$ | $3804 \pm 284$ | – | 50.0–56.5 |

Table 1: NumPy CNN on the cloud instance's CPU. "Train, 10 ep." is what the tutorial's loop costs (convolution recomputed every epoch). $b_0$ is the initial output bias; "dead at init" is the share of training images whose output starts at 0. Vectorized rows: mean $\pm$ s.d. over three seeds.

**Accuracy.** The model reaches 50.0 to 56.5% on the unseen images, depending on the seed, little better than guessing. With the tutorial's zero bias, most training images start with a negative pre-activation (99.8% for seed 0, 97.2% for seed 2), where the output ReLU passes neither a value nor a gradient. The few images that start positive pull the layer back, and by the tenth epoch at most 9.3% of images are still at 0. Removing that obstacle does not help: an initial bias of 0.5, or a linear output with no ReLU at all, gives the same 50.0–56.5% range. In the 50.0% runs every output ends on the same side of 0.5, so the model predicts a single class. The limit is the features, not the training: one random, untrained $3 \times 3$ filter followed by max-pooling keeps little that separates cats from dogs, and a linear layer on top cannot recover it.

## 3 The CNN in PyTorch on a cloud GPU

**Setup.** The GPU runs used a RunPod instance with one NVIDIA L4 (24 GB), an AMD EPYC 9254 host limited to 5.1 cores, PyTorch 2.8 with CUDA 12.8. Before measuring, a health check timed a $4096^3$ FP32 matrix multiply at 11.2 ms (12.3 TFLOP/s) with no throttling. All NumPy numbers in Section 2 were measured on the same instance's CPU.

**Model.** `CatDogCNN` follows the tutorial: two $3 \times 3$ convolutions (3→16→32 channels) each with ReLU and $2 \times 2$ max-pooling, then fully connected layers $8192 \to 128 \to 1$, ReLU on every layer including the output; MSE loss on labels 0 (cat) and 1 (dog); Adam with learning rate $10^{-3}$; batch 64; 10 epochs. The labels from `ImageFolder` are integers of shape $(B)$, while the model outputs $(B, 1)$. They must be converted to float and reshaped to $(B, 1)$; otherwise `MSELoss` broadcasts the pair to $B \times B$ and computes the wrong loss with only a warning. GPU timings call `torch.cuda.synchronize()` before reading the clock, because kernel launches return before the work is done.

**Two input pipelines.** The tutorial's pipeline is `ImageFolder` with `DataLoader`, which decodes and resizes every JPEG on the CPU every epoch (four worker processes). We also measured a "preload" pipeline that decodes the images once into a single uint8 tensor in GPU memory (7.9 s, 307 MB) and draws batches from it. The pixels are the same; only where the time goes differs.

**Accuracy.** When it trains, `CatDogCNN` reaches 77.9–80.0% on the 5,000 unseen images after 10 epochs, against 87.5–90.4% on the training images in the last epoch (Figure 2). The gap shows it is starting to overfit. It does not always train. In half of our ten runs with the tutorial's ReLU output (both pipelines, GPU and CPU), the output was 0 for every image from the first epoch on, the loss stayed at exactly 0.5, and the model labeled everything a cat. A ReLU output that reaches 0 for all inputs gets no gradient and never recovers. Replacing it with a linear output, with everything else unchanged, trained in all three seeds we tried, including the two whose ReLU runs died, and reached 77.0–78.4% (Table 2, last row). For a regression-style output trained with MSE, the output layer should not have a ReLU.

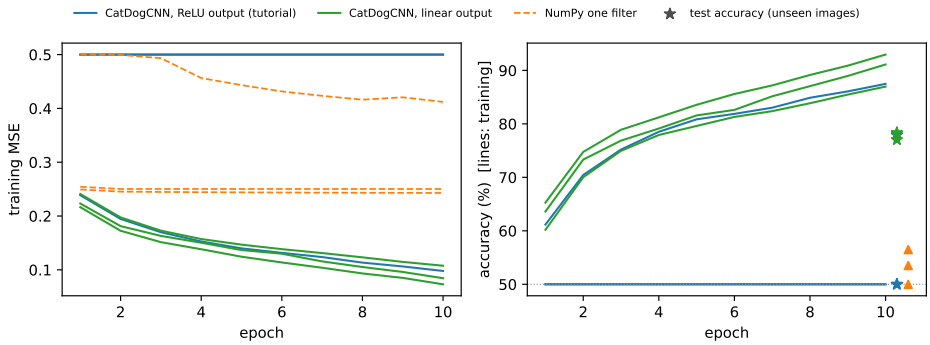



Figure 2: Training loss and accuracy per epoch on the GPU, preload pipeline, three seeds each. Blue: `CatDogCNN` as in the tutorial; two of its runs died at the start (flat at loss 0.5 and 50%). Green: the same network with a linear output. Orange: the NumPy one-filter model. Markers: accuracy on the unseen test images.

## 4 NumPy versus the GPU

Table 2 and Figure 3 put all versions side by side, with speedups relative to the tutorial's NumPy loop. The upper half compares the same one-filter model; the lower half is `CatDogCNN`.

| Model | Device | Input | Train, 10 ep. (s) | Speedup | Test (img/s) | Speedup | Test acc. (%) |
|---|---|---|---|---|---|---|---|
| 1 filter, NumPy loop | CPU | decoded | 2647 | 1× | 75 | 1× | 53.5 |
| 1 filter, NumPy vec. | CPU | decoded | 24.6 ± 0.1 | 107× | 4,039 ± 87 | 54× | 53.3 ± 3.3 |
| 1 filter, PyTorch | CPU | preload | 4.8 | 546× | 14,148 | 188× | 52.2 |
| 1 filter, PyTorch | GPU | preload | 2.1 | 1,272× | 517,860 | 6,895× | 52.2 |
| CatDogCNN | CPU | DataLoader | 141 | 19× | 1,528 | 20× | 78.0 |
| CatDogCNN | CPU | preload | 85.6 | 31× | 2,749 | 37× | 78.3 |
| CatDogCNN | GPU | DataLoader | 61.2 ± 0.4 | 43× | 2,009 ± 32 | 27× | 79.1 ± 1.3 (2/3) |
| CatDogCNN | GPU | preload | 5.1 ± 0.0 | 517× | 74,741 ± 1,391 | 995× | 77.9 (1/3) |
| CatDogCNN, linear out | GPU | preload | 5.2 ± 0.0 | 514× | 74,133 ± 2,152 | 987× | 77.9 ± 0.7 (3/3) |

Table 2: All versions on the same cloud instance, 19,998 training and 5,000 test images. Training is 10 epochs; NumPy rows count the convolution in every epoch, as the tutorial's loop does. Test throughput includes the input pipeline. GPU `CatDogCNN` rows: mean ± s.d. over three seeds; accuracy over the runs that trained.

**Same model, different hardware.** For the one-filter model, the tutorial's NumPy loop needs 2,647 s for 10 epochs and the GPU needs 2.1 s, 1,272 times less, with the same 52–54% accuracy. Most of that factor is not the GPU. Vectorizing the NumPy convolution alone gives 107×, and PyTorch on the same five CPU cores gives 546×. The GPU's own share is the last factor of 2.3: the model does so little arithmetic per image that there is not much for it to accelerate.

**CatDogCNN on CPU and GPU.** For the real network, the GPU trains 10 epochs in 5.1 s and the CPU in 85.6 s, a factor of 17, and classifies 74,700 test images per second against 2,700 (27×). Those

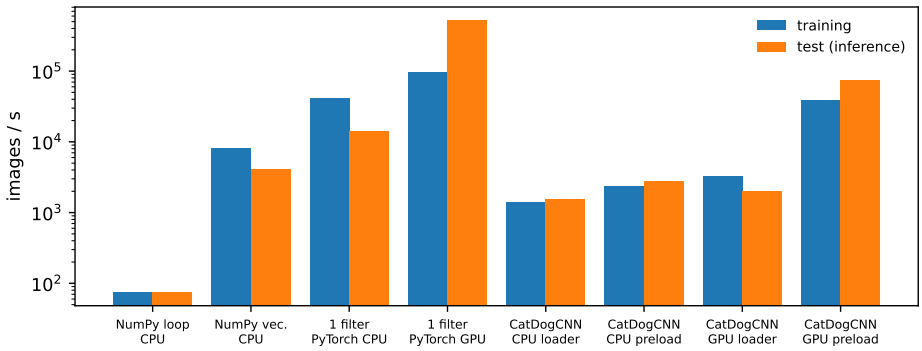



Figure 3: Training and test throughput, images per second (log scale).

are the numbers with the preload pipeline. With the tutorial's `DataLoader`, the GPU takes 61 s, of which 55 s (89%) it spends waiting for the CPU workers to decode JPEGs. The GPU is then only 2.3 times faster than the CPU with the same pipeline, and inference drops to 2,000 images per second, below the CPU's preloaded rate. Once the model is on a GPU, the input pipeline sets the speed.

**CPU threads.** PyTorch on this instance starts 24 threads, but the container is allowed 5.1 cores. With the default, `CatDogCNN` trains in 268 s on the CPU; with five threads, in 85.6 s. The CPU rows in Table 2 use five threads. The default would have made the GPU look 52 times faster instead of 17.

## A GitHub log

Commits to `lab7/` in TINKPA/ee542-group6; each message body describes the change.

| Commit | Date (PDT) | Author | Message |
|---|---|---|---|
| `6e24fde` | 2026-10-18 16:07 | Huacong Tang | feat(lab7): two-layer NumPy MLP and the three tutorial examples |
| `cf7eb32` | 2026-10-18 16:07 | Xiao Yang | feat(lab7): Dogs-vs-Cats data preparation |
| `7586e10` | 2026-10-18 16:07 | Xiao Yang | feat(lab7): one-filter NumPy CNN with block evaluation and timing |
| `c987f31` | 2026-10-18 16:07 | Jiachen Lu | feat(lab7): CatDogCNN in PyTorch on CPU and GPU |
| `40026ef` | 2026-10-18 16:07 | Huacong Tang | feat(lab7): cloud GPU campaign scripts |
| `b2bc312` | 2026-10-18 16:07 | Jiachen Lu | docs(lab7): README with how to run and the results |